

Python for AI & Machine Learning

Class 21 – Advanced NumPy Operations

Course: Python for AI & Machine Learning

Module: Data Science & Numerical Computing

Duration: 2 Hours (Theory + Practical)

Learning Objectives

By the end of this session, students will be able to:

- Reshape NumPy arrays.
 - Convert multi-dimensional arrays into one-dimensional arrays.
 - Resize arrays.
 - Understand Broadcasting.
 - Perform Boolean and Fancy Indexing.
 - Sort and Search arrays.
 - Stack and Split arrays.
 - Understand Vectorization and its importance.
 - Apply advanced NumPy operations in AI & Machine Learning projects.
-

Revision

In the previous class, we learned:

- Introduction to NumPy
- Installing and Importing NumPy
- Creating Arrays
- One-Dimensional, Two-Dimensional, and Three-Dimensional Arrays
- Array Properties (`shape` , `size` , `ndim` , `dtype`)
- Mathematical Operations
- Aggregate Functions
- Indexing and Slicing

Today, we will explore **Advanced NumPy Operations**, which are widely used in Data Science and AI.

What are Advanced NumPy Operations?

Advanced NumPy operations allow us to manipulate, organize, filter, and transform large datasets efficiently. These operations are the backbone of machine learning preprocessing and data analysis.

Array Reshaping

Reshaping changes the dimensions of an array without changing its data.

Syntax

```
array.reshape(rows, columns)
```

Example

```
import numpy as np

arr = np.array([1,2,3,4,5,6])

new_arr = arr.reshape(2,3)

print(new_arr)
```

Output

```
[[1 2 3]
 [4 5 6]]
```

Reshaping into Three Dimensions

```
import numpy as np

arr = np.arange(1,9)

new_arr = arr.reshape(2,2,2)

print(new_arr)
```

Output

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Flattening an Array

Sometimes machine learning algorithms require one-dimensional input.

Use `flatten()` to convert multi-dimensional arrays into a single dimension.

```
import numpy as np

arr = np.array([[1,2],[3,4]])

flat = arr.flatten()

print(flat)
```

Output

```
[1 2 3 4]
```

Resizing Arrays

`resize()` changes both the shape and the total size of an array.

```
import numpy as np

arr = np.array([10,20,30,40])

arr.resize((2,2))

print(arr)
```

Output

```
[[10 20]
 [30 40]]
```

Broadcasting

Broadcasting allows NumPy to perform arithmetic operations between arrays of different shapes.

Example

```
import numpy as np

arr = np.array([10,20,30])

print(arr + 5)
```

Output

```
[15 25 35]
```

Instead of adding 5 manually to each element, NumPy automatically broadcasts the value.

Broadcasting with Two Arrays

```
import numpy as np

a = np.array([[1],[2],[3]])

b = np.array([10,20,30])

print(a + b)
```

Output

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

Boolean Indexing

Boolean Indexing filters data based on conditions.

Example

```
import numpy as np

marks = np.array([45,80,65,90,30])
```

```
print(marks[marks >= 60])
```

Output

```
[80 65 90]
```

Applications:

- Finding passed students
- Filtering sales data
- Selecting qualified candidates

Fancy Indexing

Fancy Indexing allows accessing multiple elements using index arrays.

```
import numpy as np

arr = np.array([10,20,30,40,50])

print(arr[[0,2,4]])
```

Output

```
[10 30 50]
```

Sorting Arrays

```
import numpy as np

numbers = np.array([30,10,50,20])

print(np.sort(numbers))
```

Output

```
[10 20 30 50]
```

Sorting is useful for ranking, reporting, and analytics.

Searching in Arrays

Locate the index of specific values.

```
import numpy as np

numbers = np.array([10,20,30,20])

print(np.where(numbers == 20))
```

Output

```
(array([1, 3]),)
```

Stacking Arrays

Vertical Stacking

```
import numpy as np

a = np.array([1,2,3])

b = np.array([4,5,6])

print(np.vstack((a,b)))
```

Output

```
[[1 2 3]
 [4 5 6]]
```

Horizontal Stacking

```
import numpy as np

a = np.array([1,2,3])

b = np.array([4,5,6])
```

```
print(np.hstack((a,b)))
```

Output

```
[1 2 3 4 5 6]
```

Splitting Arrays

```
import numpy as np

arr = np.array([10,20,30,40,50,60])

print(np.array_split(arr,3))
```

Output

```
[array([10,20]), array([30,40]), array([50,60])]
```

Vectorization

Vectorization means performing operations on an entire array without using loops.

Traditional Python

```
numbers = [1,2,3,4]

result = []

for i in numbers:
    result.append(i*2)

print(result)
```

NumPy Vectorization

```
import numpy as np

numbers = np.array([1,2,3,4])
```

```
print(numbers * 2)
```

Output

```
[2 4 6 8]
```

Advantages:

- Faster execution
- Cleaner code
- Better performance
- Essential for AI and Data Science

AI Example 1 – Student Marks Filtering

```
import numpy as np

marks = np.array([45,76,89,55,92,68])

passed = marks[marks >= 60]

print(passed)
```

AI Example 2 – Image Processing

```
import numpy as np

image = np.array([
    [120,150],
    [180,200]
])

bright = image + 30

print(bright)
```

Broadcasting is commonly used to adjust image brightness.

AI Example 3 – Sales Analytics

```
import numpy as np

sales = np.array([25000,18000,32000,29000])

print(np.sort(sales))
```

AI Example 4 – Dataset Preparation

```
import numpy as np

dataset = np.arange(1,13)

dataset = dataset.reshape(3,4)

print(dataset)
```

Reshaping datasets is a common preprocessing step before training machine learning models.

Best Practices

- Use vectorized operations instead of loops whenever possible.
- Validate array shapes before reshaping.
- Use Boolean Indexing for filtering large datasets.
- Avoid unnecessary resizing that changes data unintentionally.
- Prefer NumPy operations over manual calculations for better performance.

Practical Exercise 1

Create an array with numbers from 1 to 12.

- Reshape it into a 3×4 matrix.
- Display its shape and dimensions.

Practical Exercise 2

Create a 4×4 matrix.

- Flatten it into a one-dimensional array.
 - Verify the output.
-

Practical Exercise 3

Create an array of student marks.

- Display students scoring above 75 using Boolean Indexing.
 - Sort the marks in ascending order.
-

Practical Exercise 4

Create two arrays.

- Perform Vertical Stacking.
 - Perform Horizontal Stacking.
 - Split the combined array into two equal parts.
-

Practical Exercise 5

Multiply every element in an array by 10 using Vectorization.

Compare it with a loop-based solution.

Assignment

AI Employee Performance Analyzer

Develop a NumPy-based application that:

- Stores employee performance scores.
- Filters employees scoring above 80.
- Sorts the scores.
- Reshapes the data into a matrix.
- Calculates:
 - Average Score
 - Highest Score

- Lowest Score
- Displays the final processed dataset.

Bonus Challenge:

Generate a random dataset of 100 employee scores using NumPy and identify the top 10 performers.

Interview Questions

1. What is Array Reshaping?
 2. What is the difference between `reshape()` and `resize()`?
 3. What is Broadcasting?
 4. What is Boolean Indexing?
 5. What is Fancy Indexing?
 6. What is Vectorization?
 7. Why is Vectorization faster than loops?
 8. Explain Vertical and Horizontal Stacking.
 9. What is the purpose of `flatten()`?
 10. How are Advanced NumPy operations used in AI and Machine Learning?
-

Summary

In today's session, we learned:

- Array Reshaping
- Flattening Arrays
- Resizing Arrays
- Broadcasting
- Boolean Indexing
- Fancy Indexing
- Sorting Arrays
- Searching Arrays
- Stacking Arrays
- Splitting Arrays
- Vectorization
- AI & Machine Learning Applications
- Practical Exercises
- Assignment

Advanced NumPy operations are essential for preparing, transforming, and analyzing data efficiently. These skills are widely used in Machine Learning, Deep Learning, Data Analytics, and Scientific Computing.

Next Class Preview

Class 22 – Introduction to Pandas

Topics to be covered:

- What is Pandas?
- Installing Pandas
- Series and DataFrames
- Creating DataFrames
- Reading CSV Files
- Reading Excel Files
- Exploring Data
- Data Selection and Filtering
- Basic Data Cleaning
- Real-world AI & Machine Learning Applications